

Project1 Test Write-up

一、使用工具分析程序并确定优化点

性能分析：

鉴于本地开发环境为 macOS，我采用了系统原生的性能采样工具 `sample` 来替代 Linux 下的 `perf` 工具进行初步的性能瓶颈定位。

- 首先，使用 `make DEBUG=1` 生成包含符号表的二进制文件。
- 随后，执行 `./everybit -l` 进行长旋转操作测试。在该进程运行期间，通过终端执行 `sample everybit 3` 抓取了调用栈数据。

```
Call graph:
850 Thread_25868988  DispatchQueue_1: com.apple.main-thread (serial)
850 start (in dyld) + 6076 [0x19c7e2b98]
850 main (in everybit) + 404 [0x1027ccdb4] main.c:80
850 timed_rotation (in everybit) + 444 [0x1027cd6e8] tests.c:291
850 testutil_rotate (in everybit) + 108 [0x1027cd3dc] tests.c:243
850 bitarray_rotate (in everybit) + 180 [0x1027cc98c] bitarray.c:196
850 bitarray_rotate_left (in everybit) + 72 [0x1027cc9e4] bitarray.c:205
623 bitarray_rotate_left_one (in everybit) + 132 [0x1027ccb54] bitarray.c:217
+ 244 bitarray_set (in everybit) + 164,176 [0x1027cc824,0x1027cc830] bitarray.c:0
+ 219 bitarray_set (in everybit) + 144,148, ... [0x1027cc810,0x1027cc814, ...] bitarray.c:174
+ 47 bitarray_set (in everybit) + 124 [0x1027cc7fc] bitarray.c:173
+ ! 47 bitmask (in everybit) + 36,24, ... [0x1027cc770,0x1027cc764, ...] bitarray.c:231
+ 37 bitarray_set (in everybit) + 212 [0x1027cc854] bitarray.c:175
+ 24 bitarray_set (in everybit) + 28,32 [0x1027cc79c,0x1027cc7a0] bitarray.c:163
+ 23 bitarray_set (in everybit) + 116 [0x1027cc7f4] bitarray.c:173
+ 17 bitarray_set (in everybit) + 156 [0x1027cc81c] bitarray.c:174
+ ! 17 bitmask (in everybit) + 36,32 [0x1027cc770,0x1027cc76c] bitarray.c:231
+ 12 bitarray_set (in everybit) + 192,196 [0x1027cc840,0x1027cc844] bitarray.c:172
154 bitarray_rotate_left_one (in everybit) + 116 [0x1027ccb44] bitarray.c:217
+ 79 bitarray_get (in everybit) + 112,144, ... [0x1027cc720,0x1027cc740, ...] bitarray.c:156
+ 42 bitarray_get (in everybit) + 120 [0x1027cc728] bitarray.c:156
+ ! 42 bitmask (in everybit) + 36,28 [0x1027cc770,0x1027cc768] bitarray.c:231
+ 33 bitarray_get (in everybit) + 28,32 [0x1027cc6cc,0x1027cc6d0] bitarray.c:146
38 bitarray_rotate_left_one (in everybit) + 64,52 [0x1027ccb10,0x1027ccb04] bitarray.c:216
33 bitarray_rotate_left_one (in everybit) + 104,124 [0x1027ccb38,0x1027ccb4c] bitarray.c:217
2 bitarray_set (in everybit) + 216 [0x1027cc858] bitarray.c:175
```

数据拆解与优化点确定：

通过AI进行分析后可以发现，在 850 个采样周期中，高达 100% 的时间消耗在 `bitarray_rotate_left` 分支。进一步拆解发现，底层的标量操作接口占据了绝大多数开销：

- `bitarray_set` 及其底层逻辑占据了 623 个采样点（约 73%）
- `bitarray_get` 占据了 154 个采样点（约 18%）

优化方向分析：

分析后不难发现，程序性能差的根源在于 `bitarray_rotate_left_one` 采用了极其低效的逐位旋转策略。因而，对于长度为 N 、移动步长为 k 的操作，时间复杂度就高达 $O(N \times k)$ 。这种设计不仅会产生庞大的函数调用栈开销，还会破坏内存访问的空间局部性。

在现代处理器架构中，为了加速大规模数据处理，通常依赖向量硬件和 SIMD（单指令多数数据流）机制来一次性吞吐宽字节数据。因此，在不考虑多线程的情况下，优化方向可以为：

1. **算法降维**：引入时间复杂度为 $O(N)$ 的“三次翻转法” $(a^R \ b^R)^R = ba$ ，这可以从根本上消除冗余的外层循环。
2. **字级并行**：将 `word` 看作多个可并行的处理单元。在堆栈分配小缓冲区，利用按位掩码一次性读取、翻转并写回多个 bit，如此一来就大幅削减了基础函数的调用频次。

二、伪代码

实验要求优化过程不能调用 `bitarray_new()` 函数进行大块动态内存分配。为了在不增加内存负担的前提下消除原始代码中低效的外层循环，我采用了时间复杂度为 $O(N)$ 的三次翻转法，由于是测试版代码且时间有限，所以我并没有对字级并行进行实现，准备在之后优化版中再进一步优化。

```
1 // 辅助函数：通过逐位交换，翻转指定区间内的位字符串
2 function reverse_bit_region(bitarray, start_idx, end_idx):
3     while start_idx < end_idx:
4         // 使用原有的基础函数获取两端的 bit
5         left_bit = bitarray_get(bitarray, start_idx)
6         right_bit = bitarray_get(bitarray, end_idx)
7
8         // 交叉写回，完成两个 bit 的位置交换
9         bitarray_set(bitarray, start_idx, right_bit)
10        bitarray_set(bitarray, end_idx, left_bit)
11
12        // 向中间逼近
13        start_idx = start_idx + 1
14        end_idx = end_idx - 1
15
16 // 主函数：旋转操作入口
17 function bitarray_rotate(bitarray, bit_offset, bit_length, shift_amount):
18     // 边界保护
19     if bit_length == 0:
20         return
21
22     // 统一将位移量转化为正向的左移量，并消除多余的旋转圈数
23     actual_shift = modulo(-shift_amount, bit_length)
24     if actual_shift == 0:
25         return
26
27     // 计算 A 部分和 B 部分的分割点
28     // A 部分长度为 actual_shift, B 部分长度为 bit_length - actual_shift
29     partA_end = bit_offset + actual_shift - 1
30     partB_end = bit_offset + bit_length - 1
31
32     // 步骤 1: 翻转 A 部分 ( $a^R$ )
33     reverse_bit_region(bitarray, bit_offset, partA_end)
34
35     // 步骤 2: 翻转 B 部分 ( $b^R$ )
36     reverse_bit_region(bitarray, partA_end + 1, partB_end)
37
```

```
38 // 步骤 3: 整体翻转 ((a^R b^R)^R)
39 reverse_bit_region(bitarray, bit_offset, partB_end)
```

三、编程过程中遇到的bug或陷阱

负数位移的模运算陷阱

接口定义的旋转参数为 `ssize_t bit_right_amount`，允许传入负数表示左旋。在 C 语言中，原生的 `%` 运算符对负数取模的结果仍为负数。但是开始时不知道这个小细节，我直接用原生 `%` 来计算真实的左移偏移量，导致翻转区间的锚点出现负数。

解决方法：

`bitarray.c` 中提供了一个安全的静态函数 `modulo()`，调用它统一了左右旋转逻辑。

四、对性能没有提升的无用功

1. 在确定使用“三次翻转法”之前，我尝试使用“环状替换法”。最初认为它每个元素只移动一次，而“三次翻转法”每个元素最多移动两次，所以会更快。但是在实际测试中发现，环状替换法需要计算集合长度和位移量的最大公约数，并且在移动数据时，其内存访问的步长是跳跃的。这样在逐 bit 访问的情况下，就会破坏空间局部性导致内存 Miss 急剧增加，实际性能反而更差。
2. 通过性能分析发现，`bitmask()` 函数被频繁调用。我尝试在堆栈区分配一个小缓冲区，预先计算并存储 0-7 索引对应的掩码结果，试图用查表法替代原有的移位计算，结果反而稍微增加了延迟。经查询后才知道，这是由于现代 CPU 的 ALU 处理简单的位移和取模运算只需 1-2 个时钟周期，而从内存读取查表数据往往需要更多周期。